

MLDiagnosis: End-to-End AI-Powered Chest X-Ray Diagnosis Platform

Abstract

MLDiagnosis is an enterprise-grade Artificial Intelligence healthcare platform developed to assist in the automated classification of chest X-ray images into two categories: **Pneumonia** and **Normal**. The system leverages deep learning techniques, explainable AI mechanisms, and clinical reporting tools to deliver reliable and interpretable predictions. Designed with scalability, privacy, and deployment efficiency in mind, MLDiagnosis supports both standard image formats and medical DICOM files while providing visual explanations and downloadable diagnostic reports. The platform is fully deployed as a serverless cloud application, making advanced AI-assisted radiology accessible through a web interface.

Introduction:

Pneumonia remains one of the leading causes of respiratory illness worldwide. Early and accurate diagnosis is critical for effective treatment; however, the interpretation of chest radiographs requires significant clinical expertise and can be time-consuming in high-volume healthcare environments.

The objective of MLDiagnosis is to develop an intelligent diagnostic assistant capable of analyzing chest X-rays, identifying pneumonia-related abnormalities, and providing interpretable results. The platform combines state-of-the-art deep learning models with explainable AI techniques to ensure transparency and trustworthiness in clinical decision-making.

Problem Statement:

Manual interpretation of chest X-rays presents several challenges:

- Limited availability of experienced radiologists in certain regions.
- High diagnostic workload in hospitals.
- Variability in interpretation between clinicians.
- Lack of transparency in traditional AI-based diagnostic systems.
- Difficulty handling hospital-standard DICOM medical images.

MLDiagnosis addresses these challenges by providing a scalable, explainable, and cloud-deployable AI solution for chest X-ray analysis.

Objectives:

The primary objectives of the project are:

- Develop a robust AI model capable of detecting pneumonia from chest X-rays.
 - Improve diagnostic reliability through model ensembling.
 - Provide explainable predictions using Grad-CAM visualizations.
 - Support hospital-standard DICOM imaging formats.
 - Ensure patient privacy through PHI anonymization.
 - Generate automated clinical reports in PDF format.
 - Deploy the complete solution as an accessible cloud application.
-

System Features

4.1 AI Model Ensembling

To improve prediction reliability, MLDiagnosis combines predictions from two independent deep learning architectures:

Custom CNN

A convolutional neural network designed specifically for pneumonia classification.

Transfer Learning ResNet18

A pretrained ResNet18 model fine-tuned on chest X-ray datasets.

The final prediction is obtained through mathematical averaging of probabilities from both models, reducing model-specific bias and improving overall robustness.

4.2 Explainable AI (Grad-CAM)

One major limitation of medical AI systems is their lack of interpretability.

MLDiagnosis addresses this by implementing **Gradient-weighted Class Activation Mapping (Grad-CAM)**, which:

- Generates heatmaps indicating regions influencing predictions.
- Highlights suspicious pulmonary areas.

- Assists clinicians in understanding model reasoning.
- Improves trust and transparency.

The generated heatmaps are overlaid on the original X-ray image for easy interpretation.

4.3 Native DICOM Support

Hospitals commonly store medical imaging data in the DICOM format.

MLDiagnosis supports direct upload of .dcm files and performs:

- DICOM image extraction.
- Metadata processing.
- Automatic PHI anonymization.
- Safe conversion to inference-ready images.

This ensures compatibility with real-world clinical workflows.

4.4 Confidence Calibration

Raw neural network probabilities are often poorly calibrated and may produce overconfident predictions.

To address this issue, the platform implements **Temperature Scaling**, which:

- Calibrates model confidence.
 - Produces clinically meaningful probabilities.
 - Improves reliability of diagnostic outputs.
 - Reduces risks associated with overconfident AI predictions.
-

4.5 Automated PDF Clinical Reports

MLDiagnosis automatically generates professional PDF reports containing:

- Original chest X-ray image.
- Grad-CAM heatmap visualization.
- Diagnostic prediction.
- Confidence score.
- Severity estimation.
- AI-generated recommendations.

These reports can be downloaded and shared for further clinical review.

4.6 Interactive Clinical Dashboard

The application provides a responsive web interface developed using Streamlit.

Dashboard features include:

- Two-column layout.
 - Image preview panels.
 - Color-coded diagnostic result cards.
 - Severity indicators.
 - Downloadable reports.
 - Real-time AI inference.
-

5. Technical Architecture

Frontend Layer

- Streamlit
- HTML/CSS Components
- Interactive Dashboard

AI Inference Layer

- PyTorch
- Torchvision
- Ensemble Prediction Engine
- Confidence Calibration Module

Explainability Layer

- Grad-CAM
- OpenCV
- Pillow

Data Processing Layer

- NumPy
- Pandas

- PyDicom

Deployment Layer

- Streamlit Community Cloud
- GitHub Version Control

6. Technologies Used

Category	Technology
Programming Language	Python 3.10
Deep Learning	PyTorch
Computer Vision	OpenCV, Pillow
Data Processing	NumPy, Pandas
Medical Imaging	PyDicom
Frontend	Streamlit
Version Control	Git, GitHub
Deployment	Streamlit Community Cloud
Experiment Tracking	MLflow
CI/CD	GitHub Actions

7. Model Performance

The performance of the trained models was evaluated using a held-out test dataset.

Model	Accuracy	Precision	Recall	F1 Score	AUC-ROC
ResNet18	97.2%	97.1%	99.2%	98.1%	99.6%
Baseline CNN	90.5%	89.2%	98.9%	93.8%	97.6%

Key Observation

The ResNet18 model achieved exceptionally high recall, which is particularly important in medical diagnostics because it minimizes false negatives and reduces the risk of missing pneumonia cases.

8. Development Challenges and Solutions

8.1 Cloud Deployment Constraints

Challenge

The initial architecture consisted of:

- FastAPI Backend
- Streamlit Frontend

Free cloud providers lacked sufficient memory to simultaneously run PyTorch and the web server.

Solution

The architecture was redesigned by integrating model loading and inference directly into Streamlit using:

```
@st.cache_resource
```

This enabled complete deployment on Streamlit Community Cloud without requiring a separate backend service.

8.2 PyTorch CPU Thread Thrashing

Challenge

Inference required over 14 seconds per image due to excessive CPU thread utilization.

Solution

Thread usage was optimized using:

```
torch.set_num_threads(4)
```

This reduced inference time from approximately 14 seconds to 1.5 seconds.

8.3 NumPy 2.0 Compatibility Issues

Challenge

Cloud deployment failed due to stricter scalar conversion rules introduced in NumPy 2.0.

Error:

TypeError:
only 0-dimensional arrays can be converted to Python scalars

Solution

Implemented safe tensor extraction:

```
float(np.squeeze(tensor))
```

ensuring compatibility across NumPy versions.

8.4 Linux Headless Environment Errors

Challenge

Streamlit Cloud environments lacked graphical dependencies required by OpenCV.

Solution

Replaced:

opencv-python

with:

opencv-python-headless

which is optimized for cloud server environments.

8.5 GitHub Secret Scanning Issues

Challenge

GitHub Push Protection blocked repository uploads due to exposed OAuth credential files.

Solution

- Removed sensitive credential files.
- Added them to .gitignore.
- Cleaned Git history and cache.
- Re-pushed a sanitized repository.

8.6 Module Resolution Failures

Challenge

Cloud deployment could not locate internal project modules.

Solution

Added dynamic repository path registration:

```
sys.path.append(
    os.path.dirname(
        os.path.dirname(
            os.path.abspath(__file__)
        )
    )
)
```

This ensured reliable module imports in isolated deployment environments.

9. MLOps and CI/CD Integration

The project follows modern MLOps practices through:

- Experiment tracking using MLflow.
- Automated testing using PyTest.
- Continuous Integration through GitHub Actions.
- Version-controlled model development.
- Automated deployment workflows.

These practices ensure reproducibility, reliability, and maintainability of the AI system.

10. Deployment

The final application is deployed online through Streamlit Community Cloud.

Live Application

MLDiagnosis allows users to:

- Upload chest X-rays.
- Upload DICOM files.
- View AI predictions.
- Analyze Grad-CAM explanations.
- Download PDF reports.

The deployment architecture is fully serverless and optimized for free-tier hosting environments.

11. Future Enhancements

Several advanced features are planned for future development.

11.1 Multi-Disease Classification

Expand from binary classification to detect:

- COVID-19
 - Tuberculosis
 - Lung Opacity
 - Other pulmonary diseases
-

11.2 Hospital PACS Integration

Develop HL7/FHIR-compliant pipelines for:

- Automated image retrieval
 - Direct PACS integration
 - Clinical workflow automation
-

11.3 Cloud Database Integration

Replace local SQLite storage with enterprise-grade databases such as:

- PostgreSQL
- Supabase
- AWS RDS

This will enable multi-user and multi-doctor environments.

11.4 Secure Authentication

Reintroduce enterprise authentication through:

- Google OAuth
 - JWT Validation
 - HTTP-only Cookies
 - Role-Based Access Control
-

12. Conclusion

MLDiagnosis demonstrates the successful integration of deep learning, explainable AI, medical imaging standards, and cloud deployment into a unified healthcare platform. Through model ensembling, Grad-CAM explainability, DICOM support, confidence calibration, and automated clinical reporting, the project delivers a practical and clinically interpretable AI-assisted diagnostic solution.

Beyond achieving high classification performance, the project addresses real-world engineering challenges related to deployment, performance optimization, compatibility, security, and scalability. The platform establishes a strong foundation for future expansion into multi-disease diagnosis, hospital integration, and enterprise healthcare applications.

The project showcases both machine learning expertise and full-stack AI engineering capabilities, making it a comprehensive demonstration of end-to-end healthcare AI system development.